fluxo

Seu evento, sem desencontros.

TURMA 3A2 · CURSO TI MOBILE · PROJETO DE SOFTWARE

Anna Eliza Cardoso Campos

Bianca Tolentino Menezes

Gabriela Coelho e Sá

João Pedro Soares Martins

Manuela Silva Laranjeira

Rebecca de Normandia Garcia

Eventos sem desencontros

PROBLEMA

Em eventos acadêmicos, quem participa não encontra as salas e se perde na programação. Para quem organiza, o controle das inscrições e dos avisos falha.

PÚBLICO-ALVO

Participantes de palestras, semanas temáticas e feiras, e as equipes que organizam esses eventos.

SOLUÇÃO

Um aplicativo que reúne os dados do evento: programação, inscrição com vagas, agenda, planta das salas e avisos, com uma área própria para o organizador.



A1 · Início

Cinco casos de uso principais

01

PARTICIPANTE

Inscrição com vagas

OBJETIVO

Garantir a vaga numa palestra sem passar do limite.

Cada inscrição tira uma vaga e o choque de horário é avisado.

02

PARTICIPANTE

Agenda personalizada

OBJETIVO

Ver só as palestras em que a pessoa está inscrita.

Montada a partir das inscrições, em ordem de horário.

03

PARTICIPANTE

Como chegar até a sala

OBJETIVO

Encontrar a sala da palestra a partir da entrada.

Cada palestra está ligada a uma sala e ao caminho até ela.

04

ORGANIZADOR

Comunicado aos inscritos

OBJETIVO

Avisar todos os inscritos de uma vez.

Cada pessoa recebe o próprio aviso, com contador na barra.

05

ORGANIZADOR

Painel do evento

OBJETIVO

Acompanhar como o evento está indo.

Ocupação, médias e ranking calculados das inscrições e avaliações.

02 · DEMONSTRAÇÃO · ATÉ 2 MIN

O Fluxo funcionando



NO VÍDEO, NESTA ORDEM

01 **Inscrição com vagas**
participante

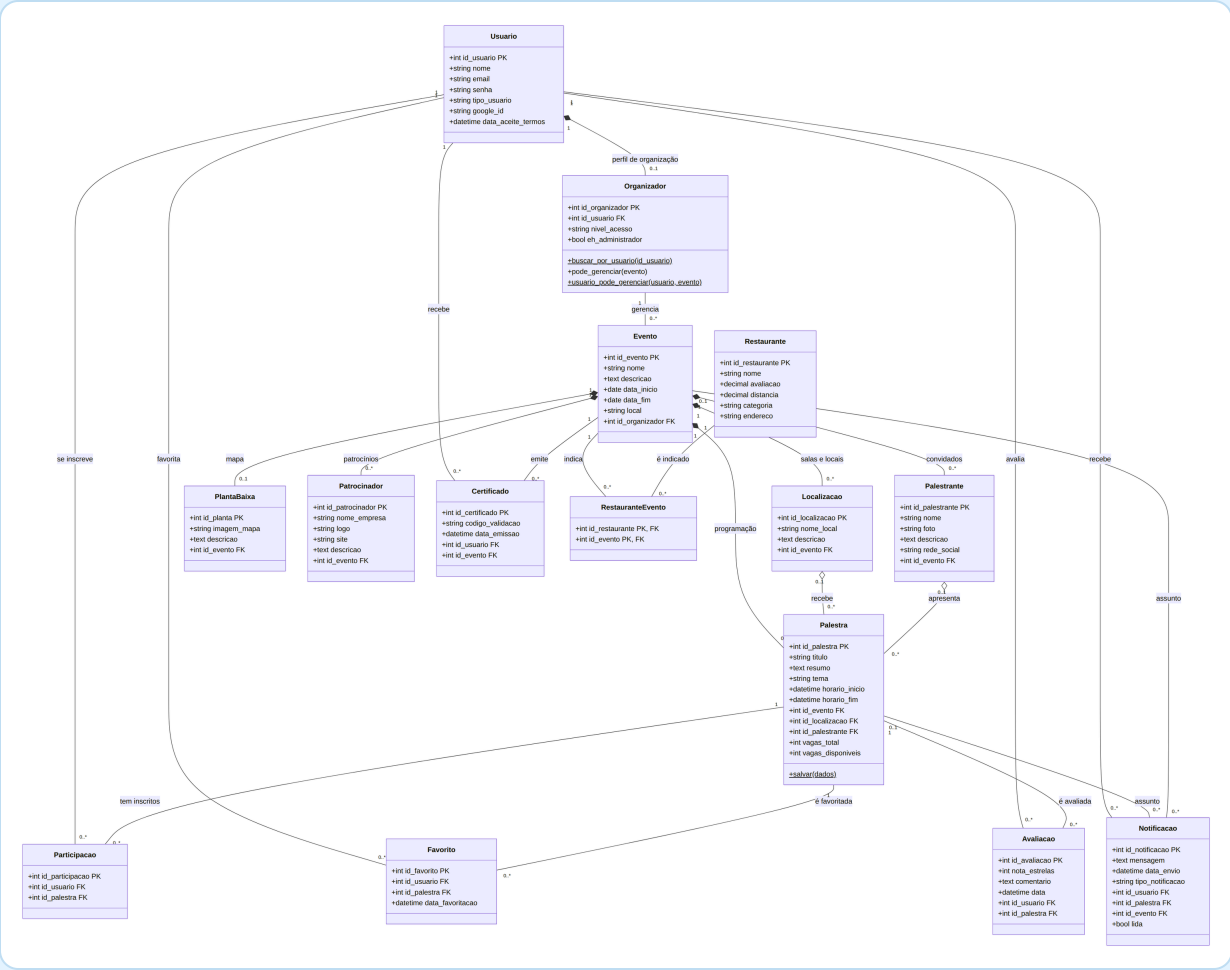
02 **Agenda personalizada**
participante

03 **Como chegar até a sala**
participante

04 **Comunicado aos inscritos**
organizador

05 **Painel do evento**
organizador

Diagrama de classes



CLASSES

15 Models. Todas herdam de db.Model e de CRUDMixin, que traz salvar(), atualizar(), deletar(), listar_todos() e buscar_por_id().

RELACIONAMENTOS

Composição: Evento ◆ Palestra. Agregação: Localizacao e Palestrante ◇ Palestra. Associação: Participacao, Favorito e Avaliacao ligam Usuario e Palestra.

CORRESPONDÊNCIA

Cada classe é um arquivo em backend/models/ e uma tabela no MySQL.

`Completo em docs/DIAGRAMA_CLASSES.md`

Ver a agenda

TELA → AgendaPage chama ApiService.agenda()

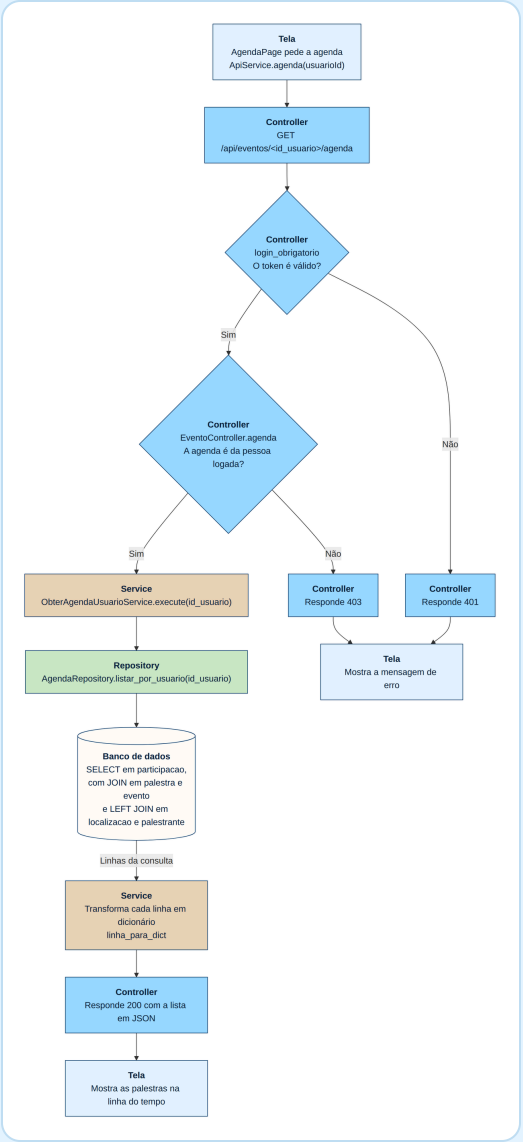
CONTROLLER → GET /api/eventos/<id>/agenda confere o token e se a agenda é da pessoa

SERVICE → ObterAgendaUsuarioService coordena o caso de uso

REPOSITORY → AgendaRepository faz o SELECT com JOIN em 5 tabelas

BANCO → MySQL devolve as linhas

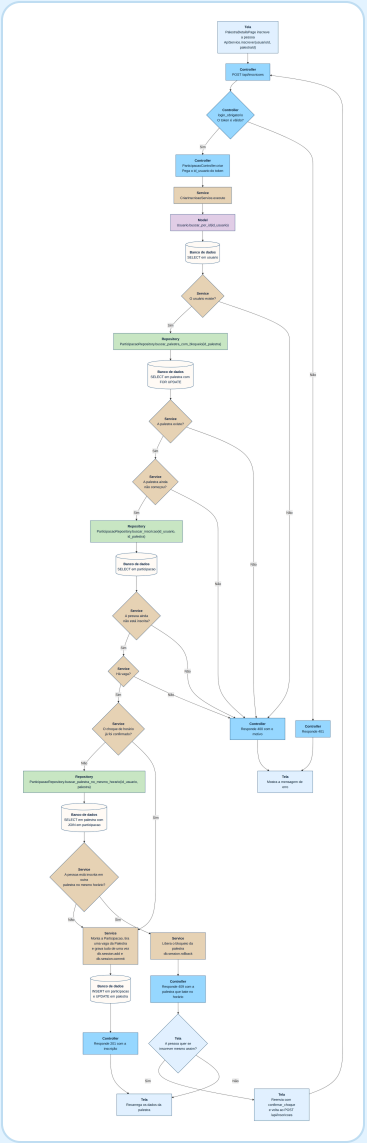
TELA → Controller responde 200 em JSON e a tela monta a linha do tempo



Completo em docs/FLUXOGRAMAS.md, com as saídas de erro

Inscrever-se em uma palestra

TELA	PalestraDetailsPage chama ApiService.inscrever()
CONTROLLER	POST /api/inscricoes confere o token
SERVICE	CriarInscricaoService confere as regras: vaga, horário, inscrição repetida
REPOSITORY	ParticipacaoRepository bloqueia a palestra (FOR UPDATE) e procura choque de horário
MODEL	Participacao é gravada e a Palestra perde uma vaga
BANCO	INSERT e UPDATE na mesma transação



Completo em docs/FLUXOGRAMAS.md, com as saídas de erro

Arquitetura e organização do código

O app e a API rodam separados, mesmo no computador local, e só conversam por HTTP.

FRONTEND

App Flutter

Celular ou emulador Android. As telas chamam o ApiService, que monta a URL a partir de API_URL.

HTTP

JSON

BACKEND · PORTA 5000

API Flask

Controllers recebem a requisição

Services um por caso de uso, com as regras

Repositories consultas SQL especiais

Models entidades e CRUD com ORM

SQLAlchemy

MySQL tabelas e dados de demonstração

PASTAS DO PROJETO

```
sistema-eventos/
├─ backend/
│  ├─ controllers/   16
│  ├─ services/      55
│  ├─ repositories/   7
│  └─ models/        16
├─ frontend/Flutter/lib/
│  ├─ pages/
│  └─ services/
│     └─ api_service.dart
└─ database/


números = arquivos em cada pasta
```

ORM no CRUD, SQL no Repository

CRUD COM ORM · SQLALCHEMY

backend/models/crud_mixin.py · herdado pelas 15 Models

`salvar(**dados)`	→ `db.session.add + commit`
`buscar_por_id(id)`	→ `db.session.get`
`listar_todos()`	→ `cls.query.all()`
`atualizar(id, **dados)`	→ `setattr + commit`
`deletar(id)`	→ `db.session.delete + commit`

USO REAL · CriarEventoService

```
Evento.salvar(nome=..., data_inicio=..., ...)
```

CONSULTA SQL NA CAMADA REPOSITORY

AgendaRepository.listar_por_usuario(id_usuario)

```
SELECT p.titulo, l.nome_local AS sala,
       p.horario_inicio
FROM participacao pa
JOIN palestra p ON ...
JOIN evento e ON ...
LEFT JOIN localizacao l ON ...
WHERE pa.id_usuario = :id_usuario
ORDER BY p.horario_inicio
```

RESULTADO PARA id_usuario = 1

titulo	sala	horario
Introdução à Inteligência Artificial	Sala A1	09:00
Banco de Dados na Prática	Sala A2	11:00

A regra: CRUD simples usa a Model (ORM). Filtro, JOIN e ranking ficam no Repository, em SQL com parâmetros.

Com o que o Fluxo foi feito

Flutter · Dart
app do celular

Python · Flask
API REST

SQLAlchemy
ORM

MySQL
banco de dados

Git · GitHub
versões e Pull Requests

Figma
protótipo inicial

Claude · Claude Code
apoio no código e na documentação

README

31

funcionalidades listadas no README, cada uma com a tela e a rota da API

Mostrado no repositório, na seção "Funcionalidades Implementadas"

BÔNUS

Termos de consentimento
O cadastro só cria a conta com o aceite dos termos, e o banco guarda a data do aceite (usuario.data_aceite_termos).

Seu evento,
sem desencontros.

O Fluxo reúne em um só aplicativo a programação, as inscrições, a localização das salas e a comunicação do evento, para participantes e organizadores.

Obrigado pela atenção.

`github.com/laranjeira1431/fluxo`